

eVTOL Simulator Technical Guide

REPORT.md 기반 시스템 구조·실행·확장 가이드

목차

각 대주제가 시작되는 페이지와 해당 장에서 확인할 수 있는 핵심 내용을 함께 정리하였다.

대주제	해설	페이지
1. 시뮬레이터 개요 및 목적 (Introduction & Objectives)	시뮬레이터의 목적, 구현 범위, 실행 경로와 파일 구성을 빠르게 파악한다.	2
2. 핵심 알고리즘 및 작동 원리 (Core Algorithms & Mathematical Principles)	좌표계, 맵 생성, 건물 회피, trajectory 생성, 충돌 판정, seed 구조를 설명한다.	5
3. 비행 물체 간 충돌 회피 메커니즘 (Collision Avoidance Mechanics)	time bucket 기반 위험 인지와 출발 지연·고도 변경 후보 선택 방식을 설명한다.	11
4. 시뮬레이션 진행 프로세스 및 실행 순서 (Execution Sequence & Process Flow)	기본 sweep, 단일 실험, 연속 시나리오, 병렬 반복 실행 흐름을 정리한다.	13
5. 우선순위 및 스케줄링 결정 로직 (Ordering & Priority Determination)	mission 처리 순서, 기체 선택, 경로 후보와 패드 선점 우선순위를 설명한다.	15
6. 핵심 컴포넌트 및 기능 레퍼런스 (Component Reference)	파일별 클래스·함수 역할과 핵심 설정 파라미터를 찾기 쉽게 정리한다.	16
7. OS별 크로스 플랫폼 실행 명령어 가이드 라인 (Multi-OS Execution Guide)	Windows와 macOS에서 바로 실행할 수 있는 설정·실행 명령어를 제공한다.	23
8. 그 외 추가로 더 설명해야 할 부분	확장 방향, 검증 절차, 성능 병목, run 수 해석 등 운영 팁을 정리한다.	27

1. 시뮬레이터 개요 및 목적 (Introduction & Objectives)

1.1 이 문서의 목적입니다

이 문서는 eVTOL 시뮬레이터를 처음 보는 연구원 또는 개발자가 코드를 빠르게 이해하고, 실행하고, 수정할 수 있도록 돕기 위한 기술 가이드입니다. 설명의 중심은 시뮬레이터의 기능, 데이터 흐름, 알고리즘, 회피 로직, 실행 방법입니다.

이 문서는 특정 산출물 작성을 위한 해석 문서가 아닙니다. 따라서 추상적인 주장보다 실제 코드가 어떻게 움직이는지, 어떤 파일을 수정해야 하는지, 어떤 값이 어떤 결과에 영향을 주는지를 중심으로 설명합니다.

1.2 시뮬레이터가 하는 일입니다

이 시뮬레이터는 격자형 도심 환경에서 여러 eVTOL 임무를 반복 생성하고, 각 기체가 건물과 다른 기체, 그리고 버티포트 패드 제약을 고려해 운항하도록 만드는 Python 기반 시뮬레이션 시스템입니다.

시뮬레이터는 다음 흐름을 다룹니다.

- 도시 맵을 생성합니다.
- 맵 안에 건물을 배치합니다.
- 일부 건물 옥상에 버티포트를 배치합니다.
- 출발 버티포트와 도착 버티포트를 골라 eVTOL 임무를 생성합니다.
- 기체가 상승, 순항, 대기, 하강 단계를 거치도록 비행 궤적(trajjectory)을 만듭니다.
- 직선 경로가 건물과 충돌할 수 있으면 우회 경유점(waypoint)을 추가합니다.
- 이미 확정된 다른 기체의 비행 궤적(trajjectory)과 가까워질 수 있으면 출발 지연 또는 고도 변경 후보를 비교합니다.
- 버티포트 패드가 이미 사용 중이면 이륙 또는 착륙을 지연합니다.
- 결과를 CSV와 그림으로 저장합니다.

1.3 현재 구현된 기능 범위입니다

현재 코드가 직접 구현하는 기능은 다음과 같습니다.

- 랜덤 시드(seed) 기반 도시 맵 생성을 수행합니다.
- 격자 셀 단위로 건물을 생성합니다.
- 건물 옥상 중심에 버티포트를 배치합니다.
- 랜덤 출발지-목적지(origin-destination) 임무를 생성합니다.
- 단발 실험 조건 반복 실험(sweep)을 실행합니다.
- 연속 임무 생성과 기체 재투입을 수행합니다.
- 경유점(waypoint) 기반 건물 회피 경로를 생성합니다.
- 시간 버킷(time bucket) 기반 비행체 간 충돌 위험을 판정합니다.

- 출발 지연 후보와 고도 레이어 후보를 비교합니다.
- 버티포트 패드 점유 구간(interval)을 예약합니다.
- 여러 시나리오를 병렬로 실행합니다.
- 장시간 반복 실행 중 중단 후 재개를 지원합니다.
- 실행 단위(run)별 결과와 집계 결과를 CSV로 저장합니다.
- 도시 맵, 비행 궤적(trajecory), 비교 그래프를 PNG로 저장합니다.
- Windows CMD, Windows PowerShell, macOS Terminal 실행을 지원합니다.

1.4 현재 구현되지 않은 기능입니다

다음 기능은 현재 코드에 직접 구현되어 있지 않습니다. 이 부분을 사용할 때는 실제 구현이 아니라 확장 후보로 이해하십시오.

- 실제 eVTOL 6-DOF 동역학은 구현되어 있지 않습니다.
- 배터리, 충전량, 에너지 소비 모델은 구현되어 있지 않습니다.
- 통신 지연, 센서 오차, GPS 오차는 구현되어 있지 않습니다.
- 기상 조건이 속도나 안전거리에 영향을 주는 로직은 구현되어 있지 않습니다.
- GIS 기반 실제 도시 데이터 로딩은 구현되어 있지 않습니다.
- 강화학습, 최적제어, MCTS 기반 회피 정책은 구현되어 있지 않습니다.
- 기체 간 수평 우회 경로 생성은 구현되어 있지 않습니다.
- 버티포트별로 서로 다른 패드 수나 처리 시간을 두는 로직은 구현되어 있지 않습니다.

`simulator/weather.py`는 현재 날씨 설정(weather config)의 상태 문자열만 반환합니다. 이 값은 실제 이동 속도, 안전거리, 경로계획에는 영향을 주지 않습니다.

1.5 핵심 실행 경로입니다

시뮬레이터에는 세 가지 주요 실행 경로가 있습니다.

- `simulator/main.py`를 실행하는 기본 조건 반복 실험(sweep) 경로입니다.
- `simulator/config/default.yaml`을 사용합니다.
- 여러 실험 조건을 한 번씩 실행합니다.
- 결과를 `outputs/`와 `figures/`에 저장합니다.
- `simulator/run_scenarios_parallel.py`를 실행하는 시나리오 병렬 경로입니다.
- `simulator/config/scenarios.yaml`을 사용합니다.
- 여러 시나리오를 프로세스 단위로 동시에 실행합니다.
- 각 시나리오마다 연속 임무 생성과 결과 저장을 수행합니다.
- `simulator/run_wallclock_parallel.py`를 실행하는 반복 병렬 경로입니다.
- `simulator/config/scenarios_v2.yaml`을 사용합니다.

- S1-S6 시나리오를 동시에 실행합니다.
- 각 시나리오 프로세스가 Model A-E 조건을 반복 수행합니다.
- `run_48h_windows.bat`와 `run_48h_windows.ps1`은 이 경로를 사용합니다.

1.6 핵심 파일 구성입니다

파일	역할입니다
<code>simulator/aircraft.py</code>	임무, 비행 궤적 샘플(trjectory sample), 비행 궤적(trjectory) 결과 구조를 정의합니다.
<code>simulator/building.py</code>	건물의 2D 평면 영역(footprint)과 높이를 정의합니다.
<code>simulator/vertiport.py</code>	건물 옥상 버티포트를 정의합니다.
<code>simulator/utlis.py</code>	좌표, 거리, 선분 교차, 보간 함수를 제공합니다.
<code>simulator/map_generator.py</code>	도시 맵, 건물, 버티포트를 생성합니다.
<code>simulator/avoidance.py</code>	건물 우회 후보점을 만들고 선택합니다.
<code>simulator/path_planner.py</code>	건물 충돌을 판정하고 경유점(waypoint) 경로를 만듭니다.
<code>simulator/mobility_model.py</code>	상승, 순항, 대기, 하강 비행 궤적 샘플(trjectory sample)을 만듭니다.
<code>simulator/collision_detector.py</code>	시간 버킷(time bucket) 기반 비행체 간 충돌 위험을 계산합니다.
<code>simulator/vertiport_scheduler.py</code>	버티포트 패드 점유와 대기 시간을 계산합니다.
<code>simulator/scheduler.py</code>	Model A-E 조건을 적용하고 최종 비행 궤적(trjectory)을 확정합니다.
<code>simulator/simulation.py</code>	단발 실험을 만들고 실행하고 요약합니다.
<code>simulator/continuous_simulation.py</code>	연속 임무 생성과 기체 재투입을 담당합니다.
<code>simulator/exporter.py</code>	결과를 CSV로 저장합니다.
<code>simulator/visualizer.py</code>	맵, 비행 궤적(trjectory), 비교 그래프를 생성합니다.
<code>simulator/config.py</code>	YAML 설정 로딩, 단위 변환, 출력(output) 폴더 생성을 담당합니다.
<code>simulator/weather.py</code>	날씨 설정(weather config) 상태를 문자열로 반환합니다.
<code>simulator/main.py</code>	기본 조건 반복 실험(sweep) 실행 진입점입니다.
<code>simulator/run_scenarios_parallel.py</code>	여러 시나리오를 병렬 실행합니다.
<code>simulator/run_wallclock_parallel.py</code>	여러 시나리오를 동시에 반복 실행합니다.
<code>scripts/analyze_wallclock_v2.py</code>	반복 실행 결과를 집계하고 그래프를 생성합니다.

2. 핵심 알고리즘 및 작동 원리 (Core Algorithms & Mathematical Principles)

2.1 좌표계는 단순 3차원 표현을 사용합니다

시뮬레이터는 2D 지도 위에 고도를 추가한 단순 3D 좌표계를 사용합니다. 이 구조는 실제 항공기 동역학을 정밀하게 계산하기보다는, 경로와 고도, 시간의 관계를 빠르게 비교하기 위해 사용됩니다.

- **Point2D(x, y)**는 수평면 좌표입니다.
- **Point3D(x, y, z)**는 수평면 좌표와 고도를 함께 갖는 좌표입니다.
- **Building(x, y, width, depth, height)**는 직사각형 건물 평면 영역(footprint)과 높이를 갖습니다.
- **Vertiport(x, y, z)**는 건물 옥상에 있는 이착륙 지점입니다.
- **AircraftMission**은 출발지, 목적지, 계획 출발 시각, 기체 id를 갖습니다.
- **AircraftTrajectory**는 최종 확정된 비행 경로와 시간 정보를 갖습니다.

2.2 거리 계산 방식입니다

2D 거리는 수평 이동 거리 계산에 사용합니다.

$$d_{2D}(a, b) = \sqrt{(a.x - b.x)^2 + (a.y - b.y)^2}$$

3D 거리는 필요할 때 공간상 거리 비교에 사용할 수 있습니다.

$$d_{3D}(a, b) = \sqrt{(a.x - b.x)^2 + (a.y - b.y)^2 + (a.z - b.z)^2}$$

현재 비행 궤적(trajjectory) 생성에서는 순항 구간의 이동 시간 계산에 주로 2D 수평거리를 사용합니다. 상승과 하강은 수직 속도 기준으로 따로 계산합니다.

2.3 보간 방식입니다

기체가 한 지점에서 다른 지점으로 이동할 때, 두 점 사이의 샘플(sample) 위치는 선형 보간으로 계산합니다.

$$\begin{aligned} P(r) = & (a.x + (b.x - a.x)r, \\ & a.y + (b.y - a.y)r, \\ & a.z + (b.z - a.z)r) \\ 0 \leq r \leq 1 \end{aligned}$$

여기서 **r**은 세그먼트(segment) 진행 비율입니다. **r=0**이면 시작점이고, **r=1**이면 도착점입니다.

2.4 선분 교차 판정 방식입니다

건물과 경로가 만나는지 확인하려면 선분과 사각형의 교차를 계산해야 합니다. 이때 `simulator/utls.py`의 `orientation`, `on_segment`, `segments_intersect`, `segment_intersects_rect`가 사용됩니다.

방향성 값은 다음 식으로 계산합니다.

```
orientation(a, b, c)
= (b.y - a.y)(c.x - b.x) - (b.x - a.x)(c.y - b.y)
```

두 선분이 일반적으로 교차하려면 다음 조건을 만족해야 합니다.

```
orientation(a, b, c) * orientation(a, b, d) < 0
and
orientation(c, d, a) * orientation(c, d, b) < 0
```

점이 같은 직선 위에 있는 예외 상황은 `on_segment`가 추가로 처리합니다. `segment_intersects_rect`는 선분이 사각형 내부에 들어가거나 사각형의 네 변 중 하나와 교차하면 참으로 판정합니다.

2.5 도시 맵 생성 방식입니다

도시 맵은 `generate_city_map`이 생성합니다. 입력은 설정 딕셔너리(dictionary)와 `random.Random` 객체입니다.

생성 순서는 다음과 같습니다.

- `map.width`, `map.height`로 전체 영역 크기를 정합니다.
- `map.grid_size`로 도시를 격자 셀로 나눕니다.
- 각 격자 셀마다 `map.building_density` 확률로 건물 생성 여부를 결정합니다.
- 건물이 생성되면 셀 내부에서 건물 크기와 위치를 정합니다.
- 건물 높이는 `map.min_building_height`와 `map.max_building_height` 범위에서 뽑습니다.
- 만들어진 건물 중 일부를 골라 버티포트를 배치합니다.

건물은 도로 영역을 정밀하게 모델링하지 않고, 격자 셀 내부에 배치되는 장애물로 표현됩니다. 이 구조는 계산을 단순하게 유지하면서도 건물 회피 로직을 테스트할 수 있게 해 줍니다.

2.6 버티포트 배치 방식입니다

버티포트는 `_place_vertiports`가 배치합니다. 이 함수는 건물 목록을 무작위로 섞은 뒤, 설정된 개수만큼 건물을 골라 해당 건물 중심에 버티포트를 둡니다.

버티포트 좌표는 다음과 같이 정해집니다.

```
vertiport.x = building.x + building.width / 2
vertiport.y = building.y + building.depth / 2
vertiport.z = building.height
```

따라서 모든 버티포트는 건물 옥상에 놓인 것으로 처리됩니다.

2.7 건물 충돌 판정 방식입니다

건물 충돌 판정은 **blocking_buildings**와 **count_building_collisions**가 담당합니다.

어떤 세그먼트(segment)가 건물을 막는다고 판정되는 조건은 다음과 같습니다.

- 현재 순항고도가 건물 높이와 수직 여유고도를 합친 값보다 낮거나 같습니다.

```
cruise_altitude <= building.height + building.vertical_margin
```

- 해당 세그먼트(segment)가 건물의 확장 사각형과 교차합니다.

```
expanded_rect = building.expanded_rect(building.horizontal_margin)
segment_intersects_rect(segment_start, segment_end, expanded_rect) == True
```

두 조건을 모두 만족하면 해당 건물은 현재 경로를 막는 건물로 취급됩니다.

count_building_collisions는 전체 경유점(waypoint) 경로의 모든 세그먼트(segment)를 검사합니다. 같은 건물이 여러 segment에서 반복 탐지되더라도 건물 id를 집합(set)으로 관리하므로 한 번만 셉니다.

2.8 건물 회피 경로 생성 방식입니다

건물 회피는 **plan_path**, **candidate_detour_points**, **choose_detour_point**가 함께 처리합니다.

처음 경로는 항상 출발지와 목적지를 잇는 직선입니다.

```
waypoints = [origin, destination]
```

건물 회피가 꺼져 있으면 직선 경로를 그대로 사용합니다. 이 경우 경로가 건물을 지나가더라도 우회하지 않습니다.

건물 회피가 켜져 있으면 다음 절차를 반복합니다.

- 현재 경유점(waypoint) 경로의 각 세그먼트(segment)를 검사합니다.
- 세그먼트(segment)를 막는 건물을 찾습니다.
- 가장 먼저 탐지된 blocking building을 대상으로 우회 후보점을 만듭니다.
- 후보점 중 다른 건물 안에 들어가지 않는 점만 남깁니다.
- **현재(current) -> 후보(candidate) -> 목적(destination)** 거리 합이 가장 짧은 점을 고릅니다.
- 고른 점을 경유점(waypoint)에 추가합니다.
- **avoidance.max_detours** 횟수에 도달하거나 더 이상 blocking building이 없을 때까지 반복합니다.

후보점은 건물 확장 사각형 주변의 8개 지점입니다. 네 모서리 방향과 네 번의 중앙 방향에 후보를 둡니다. 이 방식은 최단 경로 알고리즘은 아니지만, 구현이 단순하고 결과를 직관적으로 확인하기 쉽습니다.

2.9 비행 궤적(trajectory) 생성 방식입니다

build_trajectory는 **PlannedPath**를 실제 시간 샘플(sample)이 포함된 **AircraftTrajectory**로 변환합니다.

비행 궤적(trajectory)은 다음 단계(phase)로 나뉩니다.

- **climb**: 출발 버티포트 고도에서 순항고도까지 상승합니다.
- **cruise**: 경유점(waypoint) 경로를 따라 순항고도로 이동합니다.
- **landing_wait**: 착륙 패드가 바뀔 때 목적지 상공에서 대기합니다.
- **descend**: 순항고도에서 목적지 버티포트 고도까지 하강합니다.

상승 시간은 다음과 같습니다.

```
climb_time = max(0, cruise_altitude - origin.z) / vertical_speed
```

순항 시간은 다음과 같습니다.

```
cruise_time = horizontal_segment_distance / cruise_speed_mps
```

하강 시간은 다음과 같습니다.

```
descent_time = max(0, cruise_altitude - destination.z) / vertical_speed
```

착륙 패드 대기가 있으면 순항 종료 후 **landing_wait** 단계(phase)가 추가됩니다.

2.10 비행체 간 충돌 위험 판정 방식입니다

비행체 간 충돌 위험은 **collision_detector.py**에서 계산합니다. 핵심은 시간 버킷(time bucket)입니다.

각 비행 궤적 샘플(trajectory sample)은 다음 방식으로 버킷(bucket)에 들어갑니다.

```
bucket_id = floor(sample.time_s / conflict_time_step)
```

같은 버킷(bucket)에 들어간 샘플(sample)끼리 비교합니다. 두 샘플(sample)이 위험하다고 보는 조건은 다음과 같습니다.

```
horizontal_distance < primary_safety_distance_m
and
vertical_distance < vertical_separation_m
```

여기서 수평거리는 x-y 평면 거리이고, 수직거리는 z 좌표 차이입니다.

결과는 두 가지 관점으로 기록됩니다.

- **pair_count**는 위험에 포함된 고유 기체 쌍(unique aircraft pair) 수입니다.
- **sample_count**는 시간 버킷(time bucket) 샘플(sample) 비교에서 위험 조건이 발생한 횟수입니다.

같은 두 기체가 여러 시간대에서 가까워지면 **sample_count**는 여러 번 증가할 수 있습니다. 하지만 **pair_count**는 같은 쌍(pair)을 한 번만 셉니다.

2.11 스케줄러의 후보 비교 방식입니다

schedule_single_mission은 새 임무 하나를 확정하기 위해 여러 후보를 비교합니다.

후보는 다음 조합으로 만들어집니다.

- 출발 지연 후보입니다.
- 순항고도 후보입니다.
- 건물 회피 적용 여부입니다.
- 패드 점유 제약 적용 여부입니다.

후보 점수는 가중합이 아닙니다. 현재 코드는 다음 튜플(tuple)을 사전식으로 비교합니다.

```
score = (
    conflict_score,
    building_score,
    candidate.delay_s,
    abs(altitude - base_altitude),
    candidate
)
```

비교 우선순위는 다음과 같습니다.

- 비행체 간 충돌 위험 쌍(pair) 수가 적은 후보를 먼저 선택합니다.
- 건물 충돌 수가 적은 후보를 선택합니다.
- 출발 지연이 짧은 후보를 선택합니다.
- 기본 순항고도에서 덜 벗어난 후보를 선택합니다.

이 구조를 이해하는 것이 중요합니다. 현재 코드는 비용함수 가중치를 사용하지 않습니다. 따라서 **w_conflict**, **w_delay** 같은 가중치를 조정하는 방식은 현재 구현에 없습니다.

2.12 Model A-E의 의미입니다

Model A-E는 서로 다른 기능 조합을 빠르게 비교하기 위한 실행 모드입니다.

모델	건물 회피	비행체 간 회피	패드 점유
Model A	꺼짐	꺼짐	꺼짐
Model B	켜짐	꺼짐	꺼짐
Model C	꺼짐	켜짐	꺼짐
Model D	켜짐	켜짐	꺼짐
Model E	켜짐	켜짐	켜짐

model_flags는 모델 이름을 받아 (**building_avoidance**, **aircraft_avoidance**, **pad_occupancy**) 형태의 플래그(flag)로 바꿉니다.

2.13 패드 점유 계산 방식입니다

패드 점유는 **PadOccupancyManager**가 관리합니다. 각 버티포트는 여러 개의 패드를 가질 수 있습니다. 각 패드는 이미 예약된 시간 구간(interval) 목록을 갖습니다.

새 구간(interval)을 넣으려면 **earliest_start**가 가장 빠른 사용 가능 시각을 찾습니다.

기본 아이디어는 다음과 같습니다.

- 원하는 시작 시각을 후보 시작 시각(candidate start)로 둡니다.
- 같은 패드의 기존 예약 구간(interval)과 겹치는지 확인합니다.
- 겹치면 후보 시작 시각(candidate start)를 겹친 구간(interval)의 끝 시각으로 이동합니다.
- 더 이상 겹치지 않으면 해당 시각을 반환합니다.

이륙 패드 점유 시간은 대체로 다음 값 중 큰 값을 사용합니다.

```
takeoff_occupancy = max(pad_separation_time,
    vertical_control_height / vertical_speed)
```

착륙 패드 점유 시간은 통제구역 진입 이후 하강 시간과 turnaround time을 더해 계산합니다.

```
landing_occupancy
    = vertical_control_height / vertical_speed + turnaround_time
```

2.14 시드(seed)와 재현성입니다

단발 실험은 **_stable_seed**를 사용합니다. 기본 시드(base seed)와 실험 조건 키(key)를 합쳐 결정적 시드(deterministic seed)를 만듭니다.

```
seed = base_seed + int(md5(experiment.seed_key())[ :8], 16)
```

따라서 같은 기본 시드(base seed)와 같은 실험 조건이면 같은 도시와 같은 임무가 생성됩니다.

반복 실행 경로는 반복 회차(cycle) 번호와 모델 인덱스(model index)로 실행 단위(run) 시드(seed)를 만듭니다.

```
run_seed = seed_start + cycle * 100 + model_index
```

예를 들어 **seed_start=1000**이고 첫 반복 회차(cycle)이면 Model A-E는 각각 1000, 1001, 1002, 1003, 1004 시드(seed)를 사용합니다.

2.15 데이터가 처리되는 기본 흐름입니다

시뮬레이터 내부 데이터 흐름은 다음과 같습니다.

```

YAML 설정
-> 설정 딕셔너리(config dictionary)
-> 도시 맵(city map) 생성
-> vertiport 생성
-> 임무(mission) 생성
-> 스케줄러(scheduler)가 비행 궤적(trajecory) 확정
-> collision detector가 위험 집계
-> 요약 결과와 원시 데이터 행(summary/raw rows) 생성
-> CSV 저장
-> 그림 생성

```

각 단계는 가능한 한 plain Python 데이터 클래스(dataclass)와 딕셔너리(dictionary)를 사용합니다. 따라서 디버깅할 때 객체 구조를 출력해서 확인하기 쉽습니다.

3. 비행 물체 간 충돌 회피 메커니즘 (Collision Avoidance Mechanics)

3.1 현재 회피 방식의 성격입니다

현재 비행체 간 회피는 실시간 조종 제어가 아닙니다. 새 임무를 확정하기 전에 이미 확정된 비행 궤적(trajecory)과 비교하고, 더 안전한 출발 시각 또는 고도를 선택하는 사전 스케줄링 방식입니다.

즉, 기체가 비행 중에 즉석에서 방향을 꺾는 구조가 아닙니다. 경로가 확정되기 전에 후보를 비교하고, 그중 충돌 위험이 가장 낮은 후보를 채택합니다.

3.2 충돌 위험을 인지하는 방법입니다

스케줄러는 이미 확정된 비행 궤적(trajecory)을 **accepted_buckets**에 저장합니다.

구조는 다음과 같습니다.

```
bucket_id -> aircraft_id -> [TrajectorySample, ...]
```

새 후보 비행 궤적(trajecory)이 만들어지면 같은 시간 버킷(time bucket)에 들어 있는 기존 샘플(sample)과 비교합니다.

위험 조건은 다음과 같습니다.

```

horizontal_distance < aircraft.primary_safety_distance_m
and
vertical_distance < aircraft.vertical_separation_m

```

이 조건을 만족하면 두 기체가 같은 시간대에 충분히 가까워졌다고 판단합니다.

3.3 회피 후보는 두 종류입니다

현재 코드에서 비행체 간 충돌 위험을 줄이는 방법은 두 가지입니다.

- 출발 시각을 늦춥니다.
- **simulation.max_departure_delay**까지 후보를 만듭니다.
- **simulation.delay_step** 간격으로 후보를 증가시킵니다.

- 순향고도를 바꿉니다.
- `altitude.min_cruise_altitude`부터 `altitude.max_cruise_altitude`까지 후보를 만듭니다.
- `altitude.layer_interval` 간격으로 후보를 생성합니다.

기체 간 수평 우회 경유점(waypoint)은 현재 구현되어 있지 않습니다. 수평 경유점(waypoint) 우회는 건물 회피에만 사용됩니다.

3.4 후보 선택 순서입니다

후보마다 다음을 계산합니다.

- 건물 충돌 수입니다.
- 기존 기체와의 충돌 위험 쌍(pair) 수입니다.
- 출발 지연 시간입니다.
- 기본 고도와 차이입니다.
- 패드 대기 시간입니다.

그다음 **score** 튜플(tuple)을 기준으로 가장 작은 후보를 선택합니다.

```
score = (
    conflict_score,
    building_score,
    delay_s,
    altitude_delta,
    candidate
)
```

이 방식은 단순하지만 디버깅하기 쉽습니다. 어떤 후보가 선택되었는지 알고 싶으면 각 후보의 **conflict_score**, **building_score**, **delay_s**, **altitude_delta**를 출력하면 됩니다.

3.5 확정된 비행 궤적(accepted trajectory)이 중요한 이유입니다

스케줄러는 새 임무(mission)를 처리할 때마다 바로 **accepted** 목록과 **accepted_buckets**를 갱신합니다.

따라서 나중에 생성된 임무(mission)는 앞서 확정된 임무(mission)의 경로를 고려합니다. 반대로 이미 확정된 이전 비행 궤적(trajectory)을 뒤에서 다시 바꾸지는 않습니다.

이 구조는 선입선출에 가깝습니다. 먼저 스케줄링된 기체가 공간과 시간을 먼저 점유하고, 나중 기체가 그 조건에 맞춰 지연 또는 고도 변경을 선택합니다.

3.6 위험 지표를 해석하는 방법입니다

aircraft_collision_count는 고유 기체 쌍(unique aircraft pair) 수를 기준으로 합니다. 같은 두 기체가 여러 시간 버킷(time bucket)에서 반복적으로 가까워져도 쌍(pair) 기준으로는 한 번만 셉니다.

반복 노출 횟수까지 보고 싶으면 **aircraft_conflict_sample_count**를 확인하십시오. 이 값은 샘플(sample) 비교에서 위험 조건이 발생한 횟수입니다.

collision_risk_count는 건물 충돌 수와 기체 쌍(aircraft pair) 위험 수를 합친 지표입니다. 따라서 이 값 하나만 보지 말고, **building_collision_count**, **aircraft_collision_count**, **aircraft_conflict_sample_count**를 함께 확인하십시오.

4. 시뮬레이션 진행 프로세스 및 실행 순서 (Execution Sequence & Process Flow)

4.1 기본 조건 반복 실험(sweep) 실행 흐름입니다

`python -m simulator.main`을 실행하면 다음 순서로 진행됩니다.

```
main()
-> ensure_output_dirs()
-> load_config()
-> build_experiments()
-> 각 Experiment에 대해 run_experiment()
-> seed 생성
-> random.Random(seed) 생성
-> generate_city_map()
-> generate_missions()
-> schedule_missions()
-> detect_aircraft_conflicts()
-> summarize()
-> build_raw_rows()
-> export_results()
-> plot_city_map()
-> plot_trajectories()
-> plot_summary_graphs()
```

기본 조건 반복 실험(sweep)은 여러 실험 조건을 한 번씩 실행하는 구조입니다.

4.2 단일 실험의 내부 흐름입니다

run_experiment는 단일 실험 하나를 실행합니다.

순서는 다음과 같습니다.

- 실험 조건으로 시드(seed)를 만듭니다.
- 난수 생성기(RNG)를 생성합니다.
- 도시 맵을 생성합니다.
- 임무(mission) 목록을 생성합니다.
- 스케줄러(scheduler)가 임무(mission)를 순서대로 처리합니다.
- 비행 궤적(trajjectory) 목록을 얻습니다.
- 전체 비행 궤적(trajjectory)을 대상으로 충돌 위험을 다시 집계합니다.
- 요약 결과(summary)와 원시 데이터 행(raw rows)를 만듭니다.

4.3 연속 시나리오 실행 흐름입니다

`run_continuous_scenario`는 운항 시간(duration) 동안 임무(mission)를 반복 생성합니다.

```
time = 0
while time < duration:
    사용 가능한 기체를 선택합니다.
    출발지(origin)와 목적지(destination)를 고릅니다.
    AircraftMission을 만듭니다.
    schedule_single_mission으로 비행 궤적(trajjectory)을 확정합니다.
    기체 사용 가능 시각(available time)을 `trajectory.end_time`으로 갱신합니다.
    time += mission_interval
```

사용 가능한 기체가 있으면 그중 하나를 무작위로 선택합니다. 모든 기체가 바쁘면 가장 빨리 돌아오는 기체를 선택하고, 임무 시작 시각(mission start time)을 해당 기체의 사용 가능 시각(available time) 이후로 미룹니다.

4.4 반복 병렬 실행 흐름입니다

`run_wallclock_parallel.py`는 여러 시나리오를 동시에 실행합니다.

```
main()
-> 시나리오 설정(scenario config) 로드
-> 실행할 시나리오(scenario) 선택
-> ProcessPoolExecutor 생성
-> 시나리오(scenario)별 _run_wallclock_scenario 제출
-> 각 시나리오 작업자 프로세스(scenario worker)가 독립적으로 반복 실행
-> 시나리오별 요약 결과(summary) 저장
-> 전체 실행 단위 요약 결과(run summary) 저장
```

각 시나리오 작업자 프로세스(scenario worker) 내부에서는 다음이 반복됩니다.

```
while elapsed_time < target_seconds:
    for model in [A, B, C, D, E]:
        run_seed = seed_start + cycle * 100 + model_index
        run_continuous_scenario()
        실행 단위 요약 결과(run summary) 저장
        cycle += 1
```

`--resume` 옵션을 사용하면 기존 실행 단위(run) 요약 결과(summary)를 읽고 이어서 실행합니다.

4.5 분석 실행 흐름입니다

`scripts/analyze_wallclock_v2.py`는 반복 실행 결과를 읽고 집계합니다.

```
main()
-> configure_matplotlib()
-> ensure_summary_files()
-> build_scenario_summary()
-> build_model_summary()
-> build_block_statistics()
-> write_markdown_summary()
-> write_figures()
```

이 스크립트는 시나리오(scenario)별 지표, 모델(model)별 지표, 블록(block) 단위 변동성, 그래프를 생성합니다.

4.6 time-step 단위 갱신 방식입니다

비행 궤적 샘플(trjectory sample)은 `simulation.time_step` 간격으로 생성됩니다. 예를 들어 `time_step=10`이면 10초마다 위치 샘플(sample)이 만들어집니다.

충돌 비교는 `simulation.conflict_time_step` 간격의 버킷(bucket)으로 수행됩니다. 비행 궤적 샘플(trjectory sample)의 실제 시간이 버킷 식별자(bucket id)로 변환되고, 같은 버킷(bucket)에 있는 샘플(sample)끼리 비교됩니다.

```
bucket_id = floor(time_s / conflict_time_step)
```

따라서 시간 간격(time-step)을 줄이면 더 촘촘한 비행 궤적(trjectory)을 얻을 수 있지만 계산량이 증가합니다.

5. 우선순위 및 스케줄링 결정 로직 (Ordering & Priority Determination)

5.1 임무(mission) 처리 순서입니다

단발 실험에서는 임무(mission)를 다음 기준으로 정렬합니다.

```
(planned_start_time, mission.id)
```

계획 출발 시간이 빠른 임무(mission)를 먼저 처리합니다. 시간이 같으면 임무(mission) id가 작은 임무(mission)를 먼저 처리합니다.

연속 시나리오에서는 `mission_interval`마다 하나의 임무(mission)가 생성됩니다. 생성된 임무(mission)는 즉시 스케줄링됩니다.

5.2 기체 선택 우선순위입니다

`_select_vehicle`는 현재 요청 시점에 사용 가능한 기체를 먼저 찾습니다.

사용 가능한 기체가 있으면 그중 하나를 무작위로 선택합니다.

```
available = [vehicle_id for vehicle_id, t in enumerate(vehicle_available_at)
              if t <= request_time]
```

사용 가능한 기체가 없으면 가장 빨리 available해지는 기체를 선택합니다.

```
selected = argmin(vehicle_available_at)
```

현재 코드에는 배터리 잔량, 기체 성능, 임무 중요도, 목적지 거리 기반 기체 선택 우선순위가 없습니다.

5.3 경로 후보 우선순위입니다

경로 후보는 다음 순서로 선택됩니다.

- 비행체 간 충돌 위험이 가장 적은 후보입니다.
- 건물 충돌이 가장 적은 후보입니다.

- 출발 지연이 가장 짧은 후보입니다.
- 기본 고도와 가장 가까운 후보입니다.

이 순서는 `scheduler.py`의 `score` 튜플(tuple) 순서와 같습니다.

5.4 버티포트 패드 선점 순서입니다

패드 점유는 먼저 예약된 구간(interval)이 우선합니다. 나중에 들어온 기체는 이미 예약된 구간(interval)과 겹치지 않는 가장 빠른 시각을 찾습니다.

이 방식은 명시적 우선순위 큐(explicit priority queue)가 아닙니다. 현재 코드는 가장 빠른 가능 시각 탐색(earliest feasible time search) 방식으로 동작합니다.

5.5 모델별 스케줄링 차이입니다

Model A는 건물 회피, 비행체 간 회피, 패드 점유를 모두 사용하지 않습니다.

Model B는 건물 회피만 사용합니다.

Model C는 비행체 간 회피만 사용합니다.

Model D는 건물 회피와 비행체 간 회피를 함께 사용합니다.

Model E는 Model D 기능에 패드 점유 제약을 추가합니다.

6. 핵심 컴포넌트 및 기능 레퍼런스 (Component Reference)

6.1 `simulator/aircraft.py`입니다

AircraftMission은 하나의 임무 요청을 표현합니다. 출발 버티포트, 도착 버티포트, 계획 출발 시각, 기체 id를 가집니다.

TrajectorySample은 특정 시각의 기체 위치를 표현합니다. `aircraft_id`, `time_s`, `x`, `y`, `z`, `phase`를 가집니다.

AircraftTrajectory는 최종 확정된 비행 결과입니다. 시작 시각, 순항고도, 종료 시각, 총 거리, 우회 거리, 지연 시간, 패드 지연, 건물 충돌 수, 고도 변경 수, 샘플(sample) 목록을 가집니다.

6.2 `simulator/building.py`입니다

Building은 건물 하나를 표현합니다. `id`, `x`, `y`, `width`, `depth`, `height`를 가집니다.

center는 건물 중심 좌표를 반환합니다.

rect는 (`x_min`, `y_min`, `x_max`, `y_max`) 형태의 사각형을 반환합니다.

expanded_rect(margin)은 건물 사각형을 margin만큼 확장해서 반환합니다.

6.3 simulator/vertiport.py입니다

Vertiport는 건물 옥상 이착륙 지점을 표현합니다. **id, x, y, z, building_id**를 가집니다.
point는 **Point3D(x, y, z)**를 반환합니다.

6.4 simulator/utils.py입니다

Point2D는 2D 좌표입니다.

Point3D는 3D 좌표입니다.

clamp는 값을 최소값과 최대값 사이로 제한합니다.

distance_2d는 2D 유클리드 거리를 계산합니다.

distance_3d는 3D 유클리드 거리를 계산합니다.

point_in_rect는 점이 사각형 안에 있는지 확인합니다.

orientation은 선분 교차 판정에 필요한 방향성 값을 계산합니다.

on_segment는 collinear 점이 선분 위에 있는지 확인합니다.

segments_intersect는 두 선분이 교차하는지 확인합니다.

segment_intersects_rect는 선분과 사각형이 교차하는지 확인합니다.

interpolate_3d는 두 3D 점 사이를 선형 보간합니다.

6.5 simulator/map_generator.py입니다

CityMap은 전체 맵 상태입니다. 맵 크기, grid 크기, 도로 폭, 건물 목록, 버티포트 목록을 가집니다.

generate_city_map은 설정값과 난수 생성기(RNG)를 받아 도시 맵을 생성합니다.

_place_vertiports는 생성된 건물 중 일부를 골라 버티포트로 만듭니다.

6.6 simulator/avoidance.py입니다

candidate_detour_points는 건물 주변의 우회 후보점을 만듭니다.

point_inside_any_building은 후보점이 건물 영역 안에 들어가는지 확인합니다.

choose_detour_point는 유효한 후보점 중 이동거리 증가가 가장 작은 점을 선택합니다.

6.7 simulator/path_planner.py입니다

PlannedPath는 경로계획 결과입니다. 경유점(waypoint) 목록, 건물 위험 수, 건물 충돌 수, 경로 변경 수, 수평 이동 거리, 직선 거리 정보를 가집니다.

plan_path는 직선 경로 또는 건물 회피 경유점(waypoint) 경로를 생성합니다.

blocking_buildings는 특정 세그먼트(segment)를 막는 건물을 찾습니다.

count_building_collisions는 전체 경유점(waypoint) 경로에서 충돌 위험 건물 수를 셉니다.

path_distance는 경유점(waypoint) 폴리라인(polyline)의 총 수평거리를 계산합니다.

6.8 simulator/mobility_model.py입니다

build_trajectory는 **PlannedPath**를 시간 샘플(sample)이 있는 **AircraftTrajectory**로 변환합니다.

_append_segment는 한 세그먼트(segment)의 샘플(sample)을 시간 간격(time-step)으로 추가합니다.

_dedupe_samples는 동일 시간과 단계(phase)의 중복 샘플(sample)을 제거합니다.

6.9 simulator/collision_detector.py입니다

AircraftConflictReport는 충돌 위험 분석 결과입니다. **pair_count**, **sample_count**, **pairs**, **involved_aircraft**를 가집니다.

detect_aircraft_conflicts는 전체 비행 궤적(trajectory) 목록을 대상으로 충돌 위험을 계산합니다.

conflicts_with_accepted는 새 비행 궤적(trajectory) 후보와 이미 확정된 비행 궤적(trajectory) 목록 사이의 위험을 계산합니다.

build_conflict_buckets는 비행 궤적 샘플(trajectory sample)을 시간 버킷(time bucket) 구조로 정리합니다.

add_trajectory_to_buckets는 확정된 비행 궤적(trajectory)을 확정 버킷(accepted bucket)에 추가합니다.

conflicts_with_buckets는 새 후보와 기존 버킷(bucket) 사이의 위험을 계산합니다.

6.10 simulator/vertiport_scheduler.py입니다

PadOccupancyManager는 버티포트별 패드 예약 구간(interval)을 관리합니다.

earliest_start는 특정 버티포트에서 가장 빠른 사용 가능 시각을 찾습니다.

reserve는 패드 사용 구간(interval)을 예약합니다.

utilization_rows는 패드 사용 기록을 CSV 행(row) 형태로 반환합니다.

make_pad_manager는 설정(config) 값을 읽어 **PadOccupancyManager**를 생성합니다.

takeoff_occupancy_duration은 이륙 패드 점유 시간을 계산합니다.

landing_occupancy_duration은 착륙 패드 점유 시간을 계산합니다.

descent_to_control_zone_duration은 목적지 통제구역 상단까지 내려가는 시간을 계산합니다.

6.11 simulator/scheduler.py입니다

ScheduleState는 확정된 비행 궤적(trajectory), 충돌 위험 버킷(conflict bucket), 패드 관리자(pad manager)를 함께 보관합니다.

model_flags는 Model A-E를 기능 플래그(flag)로 바꿉니다.

create_schedule_state는 scheduling state를 초기화합니다.

schedule_missions는 임무(mission) 목록을 시간순으로 처리합니다.

schedule_single_mission은 하나의 임무(mission)에 대해 후보를 만들고 최종 비행 궤적(trajectory)을 확정합니다.

`_build_candidate`는 특정 start time과 altitude에서 경로계획(path planning)과 비행 궤적(trajecory) 생성을 수행합니다.

6.12 simulator/simulation.py입니다

`Experiment`는 단발 실험 조건을 표현합니다.

`SimulationResult`는 한 실험의 도시 맵(city map), 비행 궤적(trajecory), 충돌 위험 보고서(conflict report), 요약 결과(summary), 원시 데이터 행(raw rows)을 묶습니다.

`build_experiments`는 설정(config)의 조건 반복 실험(sweep) 설정을 실험 목록으로 바꿉니다.

`run_experiment`는 실험 하나를 실행합니다.

`generate_missions`는 버티포트 목록에서 랜덤 임무(mission)를 생성합니다.

`summarize`는 비행 궤적(trajecory)과 충돌 위험 보고서(conflict report)를 요약 결과(summary) 딕셔너리(dictionary)로 집계합니다.

`build_raw_rows`는 비행 궤적 샘플(trajecory sample)을 CSV 행(row)로 바꿉니다.

`_avg`는 빈 리스트를 안전하게 처리하는 평균 함수입니다.

`_stable_seed`는 기본 시드(base seed)와 조건 키(key)를 합쳐 결정적 시드(deterministic seed)를 만듭니다.

6.13 simulator/continuous_simulation.py입니다

`ContinuousScenarioResult`는 연속 시나리오 결과와 기체별 임무 횟수를 묶습니다.

`merge_config`는 기본 설정(base config)과 시나리오(scenario) 덮어쓰기 설정(override)을 합칩니다.

`run_continuous_scenario`는 운항 시간(duration) 동안 임무(mission)를 반복 생성하고 기체를 재투입합니다.

`_select_vehicle`는 사용할 기체를 선택합니다.

`_deep_update`는 중첩 딕셔너리(dictionary)를 재귀적으로 갱신합니다.

`_avg`는 평균을 계산합니다.

6.14 simulator/exporter.py입니다

`export_results`는 결과를 CSV로 저장합니다.

`_export_flight_results`는 비행 궤적(trajecory)별 요약 정보를 저장합니다.

`_export_pad_usage`는 패드 사용 구간(interval)을 저장합니다.

6.15 simulator/visualizer.py입니다

`plot_city_map`은 도시 맵을 이미지로 저장합니다.

`plot_trajectories`는 비행 궤적(trajecory) 경로를 이미지로 저장합니다.

`plot_summary_graphs`는 기본 조건 반복 실험(sweep) 결과 그래프를 생성합니다.

`_draw_map`은 건물과 버티포트를 그리는 공통 그리기(drawing) 함수입니다.

`_plot_metric_by_x`는 x축 변수별 평가지표(metric) 변화를 그래프합니다.

`_plot_model_comparison`은 모델별 지표를 비교합니다.

`_plot_safety_sensitivity`는 안전거리 변화에 따른 지표를 그래프합니다.

6.16 simulator/config.py입니다

`load_config`는 YAML 설정 파일을 읽습니다.

`kmh_to_mps`는 km/h를 m/s로 바꿉니다.

```
m/s = km/h / 3.6
```

`altitude_layers`는 최소 고도, 최대 고도, 레이어(layer) 간격으로 후보 고도 목록을 만듭니다.

`ensure_output_dirs`는 기본 출력(output) 폴더를 생성합니다.

6.17 simulator/weather.py입니다

`describe_weather`는 날씨 설정(weather config) 상태를 문자열로 반환합니다. 현재 이 값은 실제 비행 궤적(trajecory) 계산에는 반영되지 않습니다.

6.18 simulator/main.py입니다

`main`은 기본 조건 반복 실험(sweep) 실행 진입점입니다. 설정(config)을 읽고, 실험 목록을 만들고, 결과를 저장하고, 그림을 생성합니다.

6.19 simulator/run_scenarios_parallel.py입니다

`main`은 시나리오 병렬 실행을 시작합니다.

`_run_and_export_scenario`는 개별 시나리오를 실행하고 결과를 저장합니다.

`_write_vehicle_reuse`는 기체별 재사용 횟수를 저장합니다.

`_write_effective_config`는 실제 실행에 사용된 설정(config)을 저장합니다.

`_load_scenario_file`은 시나리오(scenario) YAML 파일을 읽습니다.

`_select_scenarios`는 실행할 시나리오를 선택합니다.

`_resolve_output_root`는 출력(output) 루트(root) 경로를 결정합니다.

`_quickenened_config`는 빠른 검증(quick test)용 설정(config)을 만듭니다.

`_parse_args`는 명령줄 옵션(CLI option)을 정의합니다.

6.20 simulator/run_wallclock_parallel.py입니다

`main`은 반복 병렬 실행을 시작합니다.

`_run_wallclock_scenario`는 시나리오(scenario) 하나를 담당하는 작업자 프로세스(worker) 함수입니다.

`_prepare_run_config`는 특정 모델(model)과 시드(seed)를 설정(config)에 주입합니다.

`_load_resume_state`는 이어서 실행할 때 기존 상태를 읽습니다.

`_load_previous_wall_time`은 이전 실행에서 이미 사용한 실제 경과 시간(wall time)을 계산합니다.

`_starting_cycle`은 다음 시작 반복 회차(cycle) 번호를 정합니다.

`_should_export_run`은 실행 단위(run) 상세 결과 저장 여부를 결정합니다.

`_write_first_figures_if_needed`는 첫 실행 단위(run)의 대표 그림을 저장합니다.

`_aggregate_scenario_runs`는 여러 실행 단위(run)를 시나리오(scenario) 단위로 집계합니다.

`_write_runs_summary`는 시나리오(scenario)별 실행 단위(run) 요약 결과(summary)를 저장합니다.

`_write_all_runs_summary`는 전체 실행 단위(run) 요약 결과(summary)를 저장합니다.

`_write_vehicle_reuse`는 기체별 재사용 횟수를 저장합니다.

`_load_yaml`은 YAML 파일을 읽습니다.

`_write_yaml`은 YAML 파일을 저장합니다.

`_select_scenarios`는 실행할 시나리오(scenario)를 선택합니다.

`_resolve_output_root`는 출력(output) 루트(root)를 결정합니다.

`_validate_save_run_details`는 상세 저장 옵션 값을 검증합니다.

`_parse_args`는 명령줄 옵션(CLI option)을 정의합니다.

6.21 scripts/analyze_wallclock_v2.py입니다

`configure_matplotlib`는 matplotlib 설정을 준비합니다.

`scenario_sort_key`는 S1, S2 같은 시나리오(scenario) 이름을 자연스러운 순서로 정렬합니다.

`main`은 분석 실행 진입점입니다.

`ensure_summary_files`는 필요한 요약 결과(summary) 파일을 확인하거나 재구성합니다.

`aggregate_scenario_from_runs`는 실행 단위 행(run rows)을 시나리오(scenario) 단위로 집계합니다.

`build_scenario_summary`는 시나리오(scenario) 요약 결과(summary) 데이터프레임(dataframe)을 만듭니다.

`build_model_summary`는 모델(model) 요약 결과(summary) 데이터프레임(dataframe)을 만듭니다.

`build_block_statistics`는 블록(block) 단위 통계를 만듭니다.

`sort_key_for_grouped`는 그룹화된 데이터프레임(grouped dataframe)(dataframe) 정렬을 돕습니다.

`write_markdown_summary`는 분석 요약 마크다운(markdown)을 저장합니다.

`dataframe_to_markdown`은 데이터프레임(dataframe)을 마크다운 표(markdown table)로 바꿉니다.

`format_cell`은 표 셀(table cell) 표시 형식을 정리합니다.

`write_figures`는 주요 그래프 생성을 총괄합니다.

`plot_completion`은 완료율 그래프를 생성합니다.

`plot_scenario_metric`은 시나리오(scenario)별 평가지표(metric) 그래프를 생성합니다.

`plot_model_metric`은 모델(model)별 평가지표(metric) 그래프를 생성합니다.

`plot_de_comparison`은 Model D와 Model E 비교 그래프를 생성합니다.

`plot_block_statistics`는 블록(block) 통계 그래프를 생성합니다.

`resolve_path`는 명령줄 경로(CLI path) 값을 **Path**로 변환합니다.

`parse_args`는 분석 스크립트 명령줄 옵션(CLI option)을 정의합니다.

6.22 핵심 설정 파라미터입니다

파라미터	의미입니다
<code>simulation.random_seed</code>	맵과 임무 생성 시드(seed)입니다.
<code>simulation.duration</code>	한 실행 단위(run)의 가상 운항 시간입니다.
<code>simulation.mission_interval</code>	새 임무(mission) 생성 간격입니다.
<code>simulation.time_step</code>	비행 궤적 샘플(trjectory sample) 간격입니다.
<code>simulation.conflict_time_step</code>	충돌 위험 버킷(bucket) 간격입니다.
<code>simulation.max_departure_delay</code>	출발 지연 후보의 최대값입니다.
<code>simulation.delay_step</code>	출발 지연 후보 간격입니다.
<code>map.width, map.height</code>	도시 맵 크기입니다.
<code>map.grid_size</code>	격자 셀 크기입니다.
<code>map.road_width</code>	도로 폭 설정값입니다.
<code>map.building_density</code>	셀별 건물 생성 확률입니다.
<code>map.min_building_height, map.max_building_height</code>	건물 높이 범위입니다.
<code>aircraft.fleet_size</code>	연속 운항 기체 수입니다.
<code>aircraft.cruise_speed_kmh</code>	순항 속도입니다.
<code>aircraft.vertical_speed</code>	상승과 하강 속도입니다.
<code>aircraft.primary_safety_distance_m</code>	수평 안전거리입니다.
<code>aircraft.vertical_separation_m</code>	수직 분리 기준입니다.
<code>aircraft.base_cruise_altitude</code>	기본 순항고도입니다.
<code>altitude.min_cruise_altitude</code>	최소 후보 고도입니다.
<code>altitude.max_cruise_altitude</code>	최대 후보 고도입니다.
<code>altitude.layer_interval</code>	고도 후보 간격입니다.

파라미터	의미입니다
<code>altitude.building_vertical_margin</code>	건물 위 수직 여유입니다.
<code>altitude.building_horizontal_margin</code>	건물 주변 수평 여유입니다.
<code>avoidance.max_detours</code>	최대 우회 경유점(waypoint) 추가 횟수입니다.
<code>avoidance.detour_margin</code>	우회 후보점 margin입니다.
<code>vertiports.count</code>	버티포트 수입니다.
<code>vertiports.pad_occupancy_enabled</code>	패드 점유 사용 여부입니다.
<code>vertiports.pad_count_per_vertiport</code>	버티포트당 패드 수입니다.
<code>vertiports.pad_separation_time</code>	패드 최소 분리 시간입니다.
<code>vertiports.turnaround_time</code>	착륙 후 처리 시간입니다.
<code>vertiports.vertical_control_height</code>	패드 통제구역 높이입니다.
<code>wallclock_runner.models</code>	반복 실행 모델 목록입니다.
<code>wallclock_runner.target_seconds</code>	반복 실행 목표 실제 시간입니다.
<code>wallclock_runner.seed_start</code>	반복 실행 시드(seed) 시작값입니다.

7. OS별 크로스 플랫폼 실행 명령어 가이드라인 (Multi-OS Execution Guide)

7.1 공통 요구사항입니다

Python 3.10 이상 사용을 권장합니다.

필수 패키지는 다음과 같습니다.

```
pandas
matplotlib
PyYAML
```

프로젝트 루트는 사용자의 실제 위치에 맞게 이동하십시오.

macOS 예시는 다음과 같습니다.

```
/Users/apple/Desktop/eVTOLPaper
```

Windows 예시는 다음과 같습니다.

```
C:\Users\<user>\Desktop\eVTOLPaper
```

7.2 Windows CMD 설정 방법입니다

프로젝트 폴더로 이동하십시오.

```
cd /d C:\Users\<user>\Desktop\eVTOLPaper
```

가상환경과 패키지를 자동으로 설정하려면 다음을 실행하십시오.

```
setup_windows.bat
```

수동으로 설정하려면 다음을 실행하십시오.

```
py -3 -m venv .venv
call .venv\Scripts\activate.bat
python -m pip install --upgrade pip
python -m pip install -r requirements_windows.txt
```

7.3 Windows CMD 빠른 검증 실행입니다

긴 실행 전에 빠른 검증(quick test)을 먼저 실행하십시오.

```
run_quick_windows.bat
```

직접 실행하려면 다음 명령을 사용하십시오.

```
python -m simulator.run_wallclock_parallel ^
--config simulator/config/scenarios_v2.yaml ^
--only S1 ^
--quick-test ^
--max-workers 1 ^
--output-root outputs/scenarios_wallclock_48h_quick ^
--overwrite ^
--save-run-details first
```

빠른 검증(quick test) 결과를 다시 분석하려면 다음을 실행하십시오.

```
python scripts/analyze_wallclock_v2.py ^
--output-root outputs/scenarios_wallclock_48h_quick ^
--figures-dir figures/wallclock_48h_quick ^
--figure-prefix quick ^
--summary-name wallclock_48h_quick_results_summary.md
```

7.4 Windows CMD 장시간 실행입니다

제공된 배치 파일을 사용하려면 다음을 실행하십시오.

```
run_48h_windows.bat
```

직접 실행하려면 다음 명령을 사용하십시오.


```
python -m simulator.run_wallclock_parallel ^
--config simulator/config/scenarios_v2.yaml ^
--target-seconds 172800 ^
--max-workers 6 ^
--output-root outputs/scenarios_wallclock_48h ^
--resume ^
--save-run-details first
```

--resume을 사용하면 기존 결과를 읽고 이어서 실행합니다.

--save-run-details first는 각 시나리오의 첫 실행 단위(run) 상세 결과만 저장합니다. 저장 공간을 줄이고 싶으면 이 옵션을 유지하십시오.

7.5 Windows PowerShell 실행입니다

PowerShell에서는 다음을 실행하십시오.

```
Set-Location -Path C:\Users\<user>\Desktop\eVTOLPaper
.\run_48h_windows.ps1
```

PowerShell 실행 정책 때문에 스크립트 실행이 막히면 다음 명령을 사용할 수 있습니다.

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

보안 정책이 있는 컴퓨터에서는 관리자 지침을 먼저 따르십시오.

7.6 Windows에서 분석만 다시 실행하는 방법입니다

이미 결과가 있으면 분석만 다시 실행할 수 있습니다.

```
analyze_48h_windows.bat
```

직접 실행하려면 다음을 사용하십시오.

```
python scripts/analyze_wallclock_v2.py ^
--output-root outputs/scenarios_wallclock_48h ^
--figures-dir figures/wallclock_48h ^
--figure-prefix 48h ^
--summary-name wallclock_48h_results_summary.md
```

7.7 macOS 설정 방법입니다

터미널에서 프로젝트 루트로 이동하십시오.

```
cd /Users/apple/Desktop/eVTOLPaper
```

가상환경을 만드십시오.

```
python3 -m venv .venv
```

가상환경을 활성화하십시오.

```
source .venv/bin/activate
```

패키지를 설치하십시오.

```
python -m pip install --upgrade pip
python -m pip install pandas matplotlib PyYAML
```

requirements 파일을 사용해도 됩니다.

```
python -m pip install -r requirements_windows.txt
```

matplotlib GUI backend 문제를 피하려면 다음을 설정하십시오.

```
export MPLBACKEND=Agg
```

UTF-8 문제를 줄이려면 다음을 설정할 수 있습니다.

```
export PYTHONUTF8=1
```

7.8 macOS 빠른 검증 실행입니다

```
python -m simulator.run_wallclock_parallel \
  --config simulator/config/scenarios_v2.yaml \
  --only S1 \
  --quick-test \
  --max-workers 1 \
  --output-root outputs/scenarios_wallclock_48h_quick \
  --overwrite \
  --save-run-details first
```

분석은 다음 명령으로 실행하십시오.

```
python scripts/analyze_wallclock_v2.py \
  --output-root outputs/scenarios_wallclock_48h_quick \
  --figures-dir figures/wallclock_48h_quick \
  --figure-prefix quick \
  --summary-name wallclock_48h_quick_results_summary.md
```

7.9 macOS 장시간 실행입니다

```
python -m simulator.run_wallclock_parallel \
  --config simulator/config/scenarios_v2.yaml \
  --target-seconds 172800 \
  --max-workers 6 \
  --output-root outputs/scenarios_wallclock_48h \
  --resume \
  --save-run-details first
```

분석은 다음 명령으로 실행하십시오.

```
python scripts/analyze_wallclock_v2.py \
--output-root outputs/scenarios_wallclock_48h \
--figures-dir figures/wallclock_48h \
--figure-prefix 48h \
--summary-name wallclock_48h_results_summary.md
```

7.10 macOS 기본 조건 반복 실험(sweep) 실행입니다

```
python -m simulator.main
```

7.11 macOS 시나리오 병렬 실행입니다

```
python -m simulator.run_scenarios_parallel \
--config simulator/config/scenarios.yaml \
--max-workers 5 \
--output-root outputs/scenarios
```

7.12 OS별 주의사항입니다

Windows CMD에서는 줄바꿈 continuation으로 ^를 사용하십시오.

PowerShell에서는 줄바꿈 continuation으로 backtick을 사용하십시오.

macOS와 Linux에서는 줄바꿈 continuation으로 \를 사용하십시오.

Windows 경로 구분자는 보통 \입니다.

macOS와 Linux 경로 구분자는 /입니다.

Python 코드 내부에서는 **pathlib.Path**를 많이 사용하므로 대부분의 경로는 OS 독립적으로 처리됩니다. 다만 **.bat**와 **.ps1** 파일은 Windows 전용입니다.

7.13 CPU와 저장 공간 주의사항입니다

--max-workers 6을 사용하면 S1-S6 여섯 개 시나리오가 동시에 실행됩니다. CPU core가 부족하면

--max-workers 3 또는 **--max-workers 4**로 줄이십시오.

--save-run-details all은 모든 실행 단위(run)의 상세 비행 궤적(trajecory)을 저장합니다. 이 옵션은 저장 공간을 많이 사용합니다. 일반적으로 **first** 또는 **none**을 권장합니다.

8. 그 외 추가로 더 설명해야 할 부분

8.1 이 시뮬레이터를 이해할 때 가장 중요한 관점입니다

이 시뮬레이터는 다음 요소를 한 번에 실행해 볼 수 있는 실무형 테스트 환경입니다.

- 도시 격자 맵을 생성합니다.
- 랜덤 건물을 생성합니다.
- 옥상 버티포트를 생성합니다.
- 연속 임무를 생성합니다.

- 기체를 재투입합니다.
- 건물 회피를 수행합니다.
- 비행체 간 사전 충돌 회피 조정(de-confliction)을 수행합니다.
- 버티포트 패드 점유를 계산합니다.
- Model A-E 기능 조합을 비교할 수 있습니다.
- 반복 실행 단위(run) 결과를 집계할 수 있습니다.

새 기능을 붙일 때는 먼저 어느 단계에 개입할지 정하십시오. 예를 들어 도시 생성 방식을 바꾸려면 `map_generator.py`부터 보십시오. 기체 간 회피 정책을 바꾸려면 `scheduler.py`와 `collision_detector.py`를 먼저 보십시오.

8.2 Model E를 해석하는 방법입니다

Model E는 Model D에 패드 점유 제약을 추가한 모델입니다. 따라서 Model E는 항상 더 좋은 결과를 내도록 만든 모델이 아닙니다.

패드 점유 제약이 켜지면 이륙과 착륙 대기가 생길 수 있습니다. 이 때문에 완료율이나 지연 시간이 달라질 수 있습니다. Model E를 볼 때는 충돌 위험뿐 아니라 `avg_pad_delay_s`, `max_pad_delay_s`, `pad_wait_flight_count`를 함께 확인하십시오.

8.3 실패 또는 미완료 상태를 구분하는 방법입니다

`failed_flights`는 충돌 위험에 포함된 기체 수를 기반으로 합니다.

`completed_after_duration`은 운항 시간(duration)이 끝난 뒤에 완료된 임무 수입니다. 이 값은 곧바로 충돌 실패를 뜻하지 않습니다.

상태를 더 세분화하고 싶으면 다음 항목을 추가하십시오.

- 건물 충돌 잔존 여부입니다.
- 비행체 간 쌍(pair) 위험 포함 여부입니다.
- 운항 시간(duration) 내 완료 여부입니다.
- 패드 대기 초과 여부입니다.
- 경로 생성 실패 여부입니다.
- 고도 후보 부족 여부입니다.

8.4 통계 확인을 확장하는 방법입니다

현재 `scripts/analyze_wallclock_v2.py`는 블록(block) 단위 통계를 계산합니다. 더 자세한 변동성을 보고 싶으면 다음 지표를 추가하십시오.

- 시나리오(scenario)별 평균을 계산하십시오.
- 시나리오(scenario)별 표준편차를 계산하십시오.
- 시나리오(scenario)별 표준오차를 계산하십시오.
- bootstrap 기반 구간을 계산하십시오.

- 모델별 지표 차이를 별도 table로 만드십시오.

8.5 민감도 분석을 추가하는 방법입니다

민감도 분석에 적합한 설정값은 다음과 같습니다.

- `vertiports.pad_count_per_vertiport`
- `vertiports.turnaround_time`
- `vertiports.pad_separation_time`
- `simulation.mission_interval`
- `aircraft.fleet_size`
- `aircraft.primary_safety_distance_m`
- `aircraft.vertical_separation_m`
- `altitude.layer_interval`
- `altitude.min_cruise_altitude`
- `altitude.max_cruise_altitude`
- `avoidance.detour_margin`
- `avoidance.max_detours`

설정 파일을 복사한 뒤 한 번에 하나의 파라미터만 바꿔 비교하면 원인을 확인하기 쉽습니다.

8.6 날씨(weather) 로직을 추가하는 방법입니다

현재 weather는 운항 계산에 영향을 주지 않습니다. 실제 영향을 넣으려면 다음 파일을 수정하십시오.

- `weather.py`에서 조건(condition)별 속도 보정, 안전거리 보정, 운항 가능 여부를 반환하십시오.
- `continuous_simulation.py`와 `simulation.py`에서 속도(speed) 계산 후 날씨 보정(weather correction)을 적용하십시오.
- `scheduler.py`에서 안전거리, 고도 후보, 지연(delay) 후보에 날씨(weather) 영향을 넣으십시오.
- `summarize`에서 날씨 조건(weather condition)과 보정 계수를 요약 결과(summary)에 기록하십시오.

8.7 실제 도시 데이터를 넣는 방법입니다

현재 맵 생성기(map generator)는 합성 격자 도시(synthetic grid city)를 만듭니다. 실제 도시 데이터를 쓰려면 다음을 추가하십시오.

- `map_generator.py`에 GIS 또는 CSV 기반 building 로더(loader)를 추가하십시오.
- `Building`에 실제 건물 id, 용도, 높이 속성을 추가하십시오.
- `Vertiport`를 랜덤 배치가 아니라 지정 좌표 기반으로 만들 수 있게 하십시오.
- 도로 폭과 grid size 개념을 실제 도로망 데이터로 대체하십시오.

8.8 경로계획을 확장하는 방법입니다

현재 건물 회피는 탐욕적 경유점(greedy waypoint) 방식입니다. 더 복잡한 경로계획을 원하면 다음 방법을 검토하십시오.

- A* 격자 라우팅(grid routing)을 추가할 수 있습니다.
- 가시성 그래프(visibility graph)를 추가할 수 있습니다.
- RRT 또는 RRT*를 추가할 수 있습니다.
- 3D 복셀 탐색(voxel search)을 추가할 수 있습니다.
- 에너지 고려 경로계획(energy-aware path planning)을 추가할 수 있습니다.

수정 위치는 `path_planner.py`와 `avoidance.py`입니다.

8.9 충돌 회피를 확장하는 방법입니다

현재 비행체 간 회피는 출발 지연과 고도 레이어 변경을 사용합니다. 더 다양한 회피를 원하면 다음을 추가하십시오.

- 수평 우회 경유점(waypoint)을 추가하십시오.
- 속도 조절 후보를 추가하십시오.
- 목적지 거리 기반 우선순위(priority)를 추가하십시오.
- 긴급 임무 우선순위(priority)를 추가하십시오.
- 롤링 호라이즌 스케줄링(rolling horizon scheduling)을 추가하십시오.
- 충돌 심각도 점수(conflict severity score)를 추가하십시오.

주요 수정 위치는 `scheduler.py`의 `schedule_single_mission`입니다.

8.10 버티포트 운영을 확장하는 방법입니다

현재 버티포트는 같은 패드 수(pad count)와 같은 처리 시간(turnaround time)을 사용합니다. 현실적인 운영 조건을 넣으려면 다음을 추가하십시오.

- 버티포트별 패드 수(pad count)를 추가하십시오.
- 버티포트별 처리 시간(turnaround time)을 추가하십시오.
- 충전 용량을 추가하십시오.
- 승객 처리 시간을 추가하십시오.
- 대기열 길이(queue length) 제한을 추가하십시오.
- 도착 우선순위와 출발 우선순위를 분리하십시오.
- 이륙 패드와 착륙 패드를 분리하십시오.

수정 위치는 `vertiport_scheduler.py`, `scheduler.py`, 시나리오(scenario) YAML 파일입니다.

8.11 성능 병목을 줄이는 방법입니다

현재 충돌 비교는 시간 버킷(time bucket)을 사용하지만, 같은 버킷(bucket) 안에서는 샘플(sample) 쌍을 모두 비교합니다. 기체 수와 샘플(sample) 수가 커지면 계산량이 커질 수 있습니다.

개선하려면 다음을 검토하십시오.

- 공간 해싱(spatial hashing)을 추가하십시오.
- KD-tree 또는 격자 인덱스(grid index)를 추가하십시오.
- 고도 레이어(altitude layer)별 버킷(bucket)을 분리하십시오.
- 확정 버킷(accepted bucket) 저장 구조를 최적화하십시오.
- 샘플(sample) 구간(interval)을 상황에 따라 조정하십시오.

8.12 수정 후 최소 검증 절차입니다

코드를 수정한 뒤에는 다음을 확인하십시오.

- 빠른 검증(quick test)을 실행하십시오.
- `summary_results.csv` 또는 `scenario_runs_summary.csv`가 생성되는지 확인하십시오.
- `collision_risk_count`, `aircraft_collision_count`, `aircraft_conflict_sample_count`의 의미를 구분해서 확인하십시오.
- `generated_missions = duration / mission_interval` 관계가 맞는지 확인하십시오.
- `completed_within_duration + completed_after_duration = generated_missions` 관계가 맞는지 확인하십시오.
- 패드 점유를 켜 경우 `avg_pad_delay_s`와 `pad_wait_flight_count`가 기록되는지 확인하십시오.
- `--resume` 재시작 시 실행 단위(run) id가 중복되지 않는지 확인하십시오.
- 모델별 실행 단위(run) 수가 크게 다르면 마지막 반복 회차(cycle)가 중간에 끝났는지 확인하십시오.

8.13 목적별로 먼저 볼 파일입니다

하고 싶은 작업	먼저 볼 파일입니다
도시 생성 방식을 바꾸고 싶습니다.	<code>map_generator.py</code>
건물 회피 알고리즘을 바꾸고 싶습니다.	<code>path_planner.py</code> , <code>avoidance.py</code>
비행 궤적 모델을 바꾸고 싶습니다.	<code>mobility_model.py</code>
비행체 간 회피 정책을 바꾸고 싶습니다.	<code>scheduler.py</code> , <code>collision_detector.py</code>
패드 점유 방식을 바꾸고 싶습니다.	<code>vertiport_scheduler.py</code> , <code>scheduler.py</code>
임무 생성 수요 패턴을 바꾸고 싶습니다.	<code>continuous_simulation.py</code>
반복 실행 방식을 바꾸고 싶습니다.	<code>run_wallclock_parallel.py</code>
결과 지표를 추가하고 싶습니다.	<code>simulation.py</code> , <code>exporter.py</code>

하고 싶은 작업	먼저 볼 파일입니다
그래프와 분석을 추가하고 싶습니다.	<code>scripts/analyze_wallclock_v2.py</code>
시나리오 조건을 바꾸고 싶습니다.	<code>simulator/config/scenarios_v2.yaml</code>

8.14 반복 실행 결과의 실행 단위(run) 수를 이해하는 방법입니다

`run_count`는 사람이 프로그램을 그 횟수만큼 실행했다는 뜻이 아닙니다. `run_wallclock_parallel.py`가 각 시나리오 작업자 프로세스(scenario worker) 안에서 반복 수행한 개별 시뮬레이션 실행 단위(run) 수입니다.

각 실행 단위(run)는 설정된 `duration`과 `mission_interval`에 따라 일정 수의 임무(mission)를 생성합니다.

예를 들어 다음 관계가 성립합니다.

```
missions_per_run = duration / mission_interval
total_generated_missions = total_run_count * missions_per_run
```

시나리오별 실행 단위(run) 수는 반드시 같을 필요가 없습니다. 각 시나리오마다 맵 크기, 건물 수, 기체 수, 충돌 비교량이 다르기 때문에 실행 단위(run) 하나를 계산하는 시간이 달라질 수 있습니다.

8.15 문서와 코드가 헛갈리기 쉬운 부분입니다

위험 이벤트를 볼 때는 고유 쌍(unique pair) 지표와 샘플(sample) 반복 지표를 구분하십시오.

`aircraft_collision_count`는 고유 쌍(unique pair) 중심입니다.

`aircraft_conflict_sample_count`는 시간 버킷(time bucket) 샘플(sample) 비교에서 위험 조건이 발생한 횟수입니다.

또한 현재 후보 선택은 가중치 기반 비용함수가 아닙니다. 현재는 튜플(tuple) 사전식 비교입니다. 가중치 기반 비용함수를 추가하려면 `scheduler.py`에서 점수(score) 계산 방식을 바꾸어야 합니다.

예시는 다음과 같습니다.

```
weighted_score
= w_conflict * conflict_score
+ w_building * building_score
+ w_delay * delay
+ w_altitude * altitude_delta
```

이 방식을 추가하면 기존 결과와 선택 기준이 달라집니다. 따라서 새 점수(score) 방식을 추가할 때는 기존 튜플(tuple) 방식과 별도 옵션으로 분리하는 것을 권장합니다.